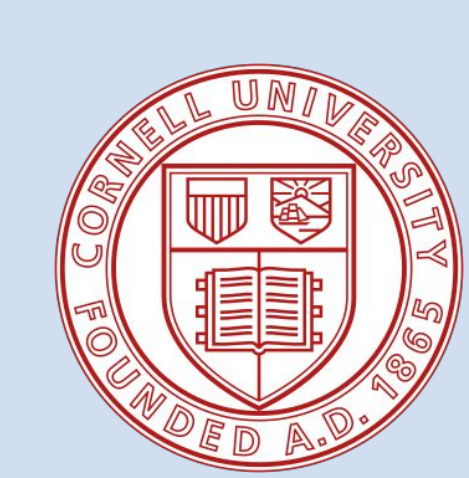




Thinking in Patches: A Better Transformer for Time Series

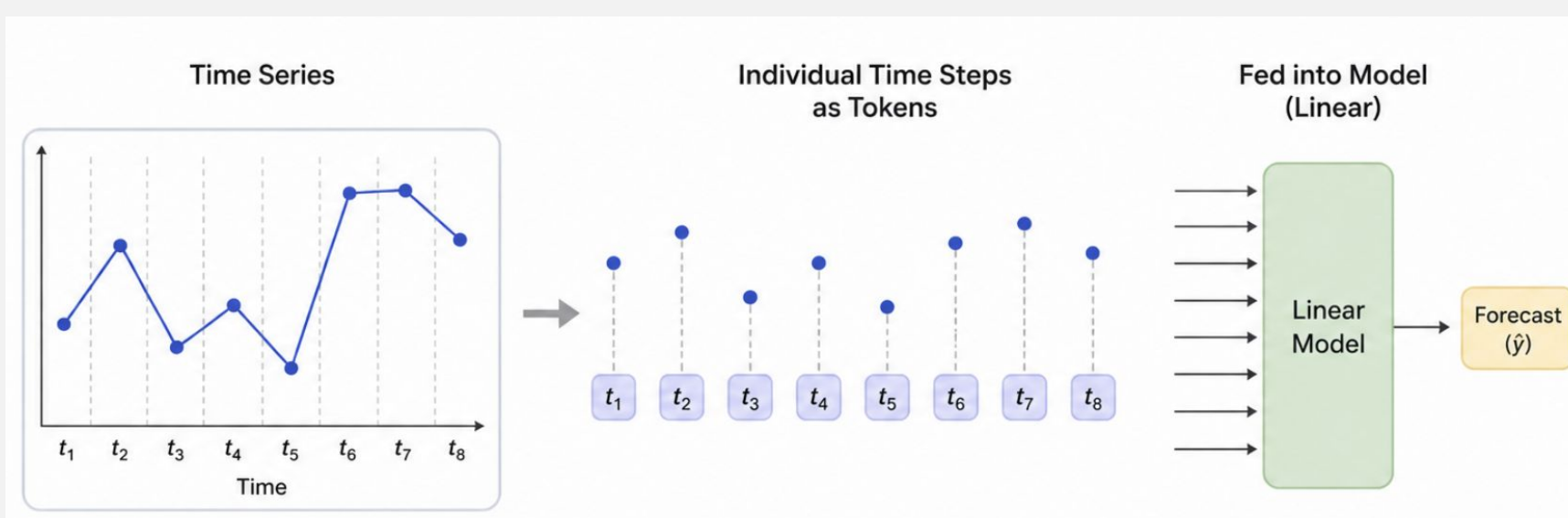
Vinay Ivaturi, Aneesh Naresh, Arnav Kaul, Jay Talwar



Problem Statement

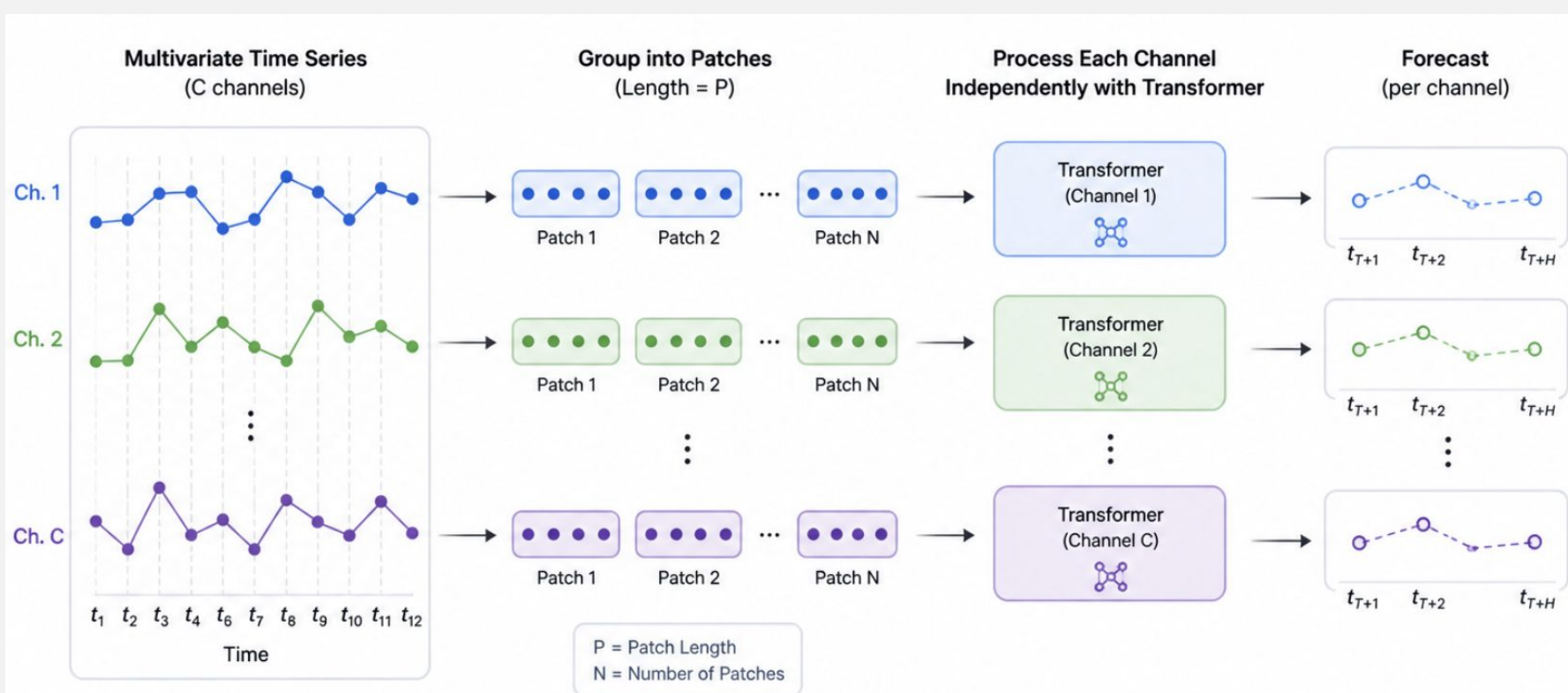
Transformers were being outperformed by a simple linear model on time series forecasting, questioning their value entirely.

Motivation



Previous models fed individual time steps as tokens, but a single timestamp carries no meaningful information alone.

Methodology



PatchTST groups time steps into patches and processes each channel independently, restoring Transformer's edge over linear models.

Datasets

Weather, Traffic, Electricity Transformer, Electricity Consumption (7-862 features)

Model Architecture

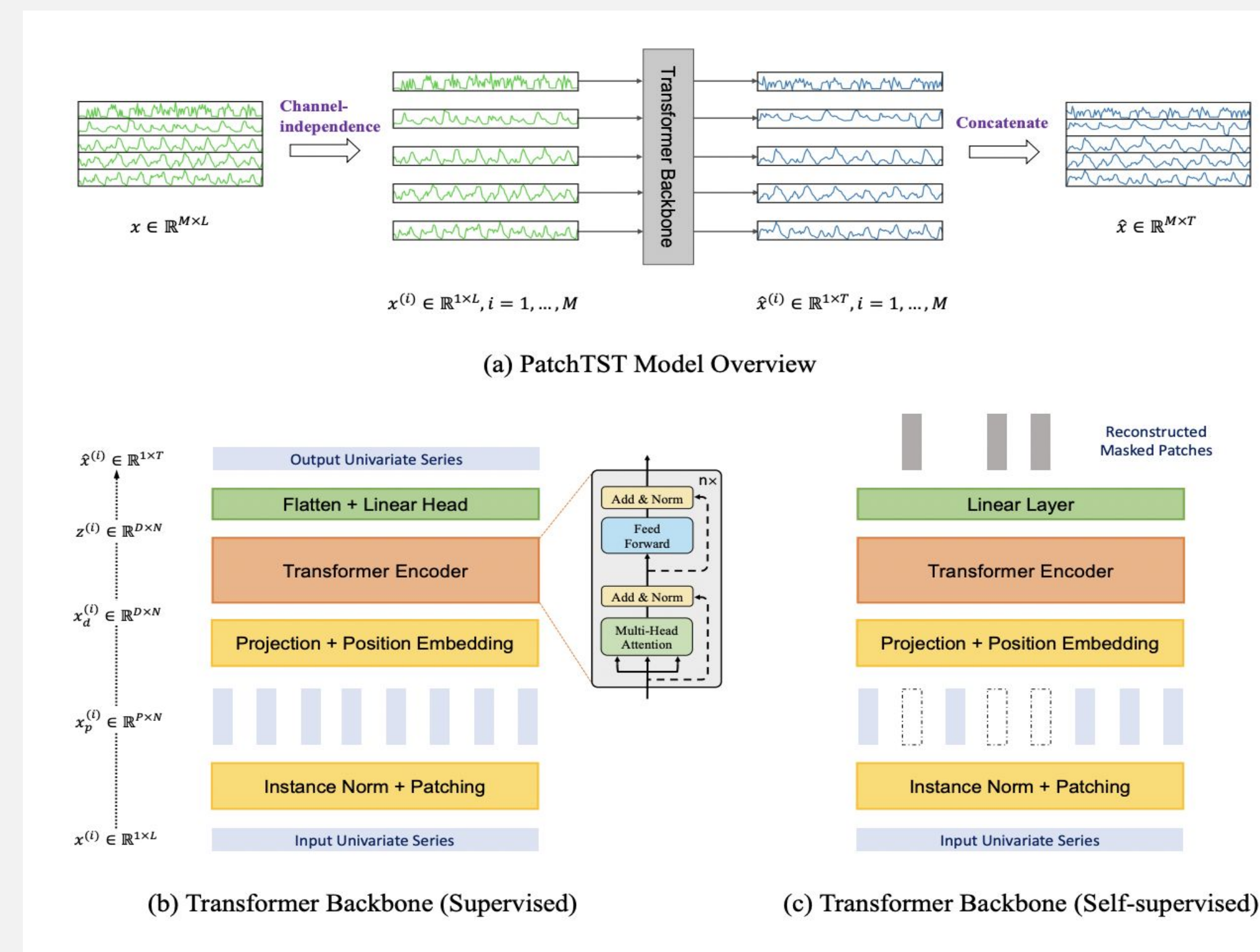


Figure 1 from Nie et al. (2023): PatchTST Architecture

Extensions

MSE on ETTh1 with varying prediction lengths

T	Paper	Base	RoPE	Channel Attention	Adaptive Patching
96	0.375	0.3797	0.3753	0.3912	0.3759
192	0.414	0.4276	0.4157	0.442	0.4171
336	0.431	0.4665	0.4401	0.4893	0.4418
720	0.448	0.4813	0.456	0.574	0.4579

Channel attention (replaces strict channel independence), **Adaptive patching** (extends fixed patching), **Time-series decomposition**, **Rotary Positional Embeddings (RoPE)**, **RevIN + affine normalization**, **Batch/Layer normalization**

Results

Paper

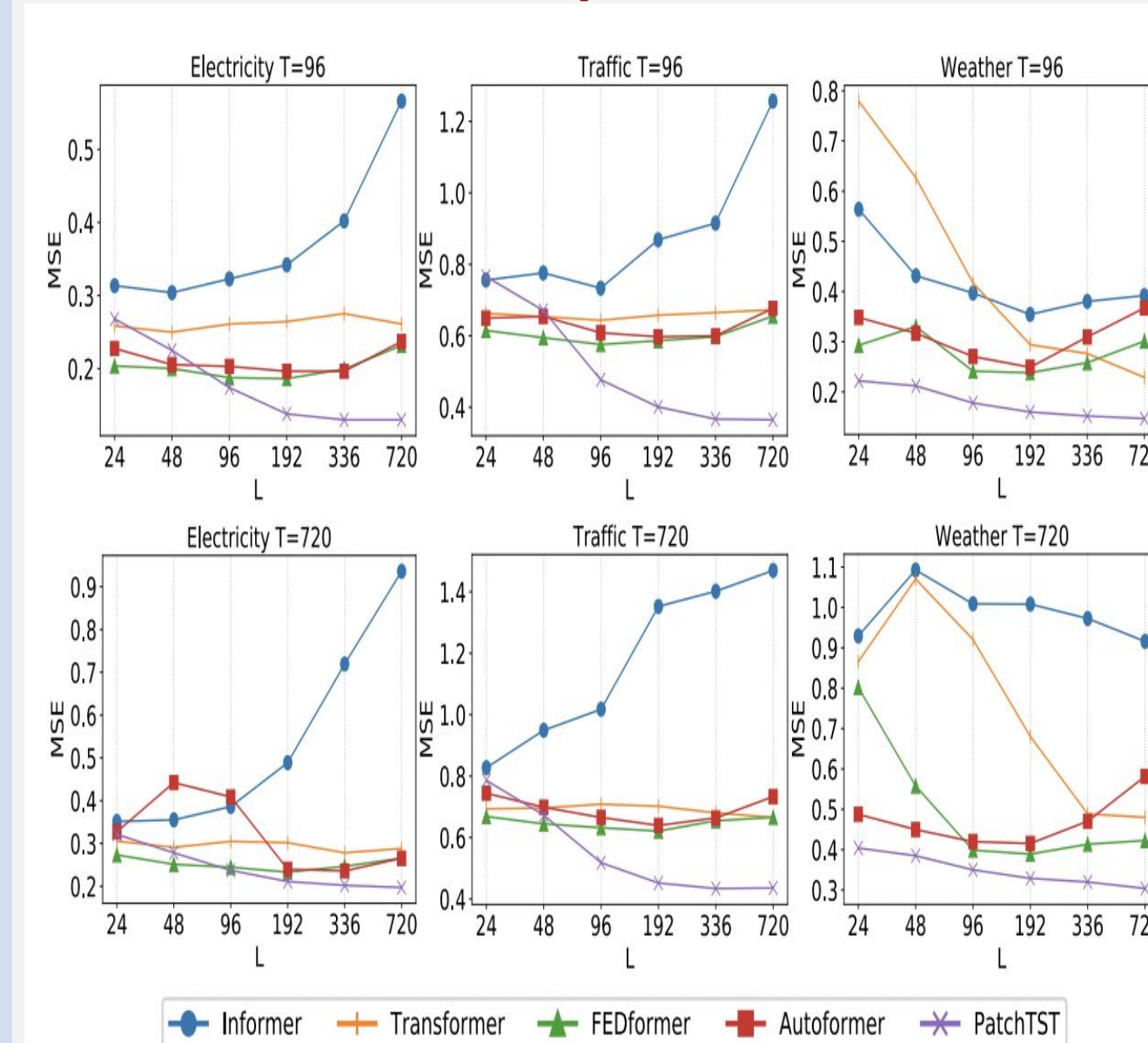
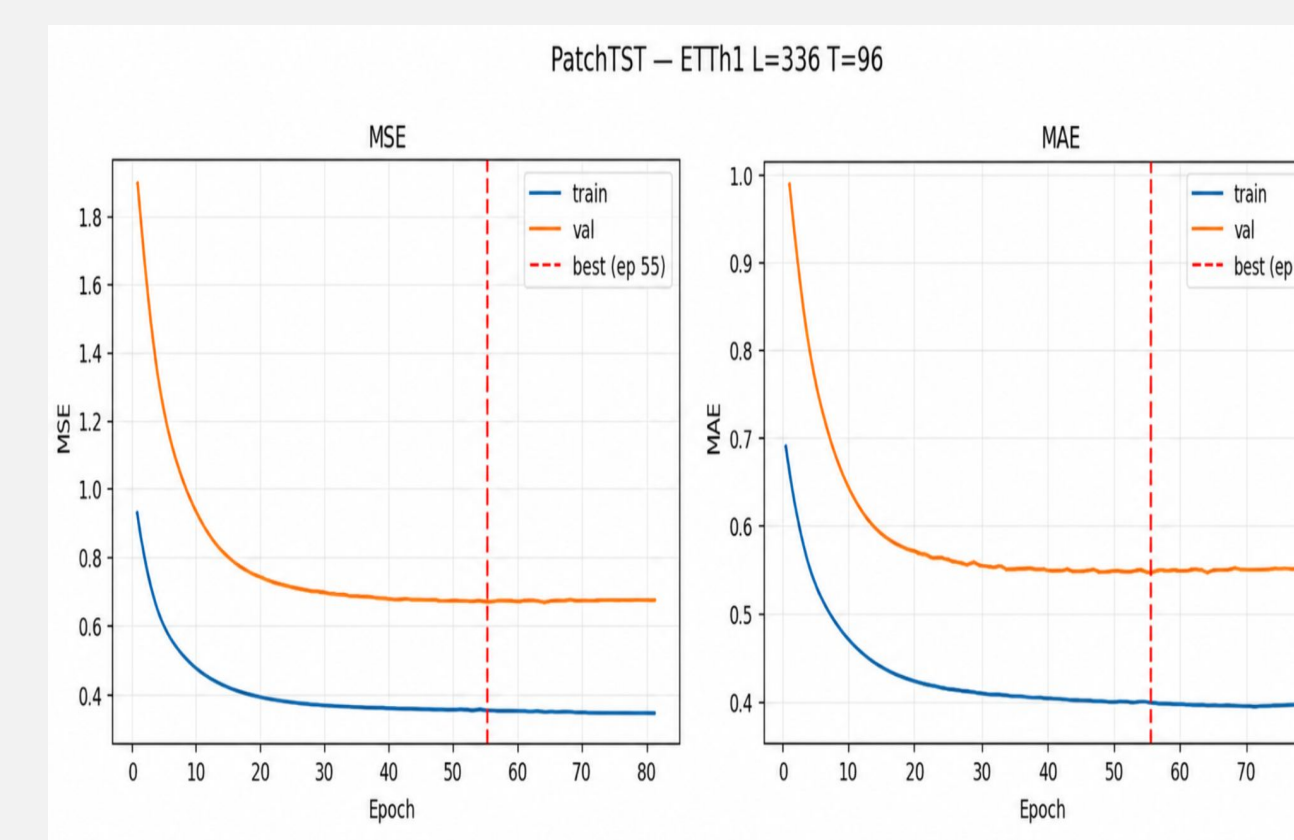


Figure 2 from Nie et al. (2023): Forecasting performance (MSE) with varying look-back windows on 3 large datasets: Electricity, Traffic, and Weather

Our Replication



Our replication: train vs. validation loss on ETTh1 dataset (L=336, T=96)

Conclusion

Key Takeaway: PatchTST outperforms all baseline models across all look-back windows, proving Transformers are effective for time series when designed correctly.

Limitations

- Ignores cross-channel correlations by design
- Less stable on smaller datasets
- Tested only on academic benchmarks

Future Work

- Evaluate cross-channel attention on different, more complex datasets
- Implement hierarchical patching to better model short and long-term patterns